

COMP 8005

Assignment 2

Design

Winston Nguyen
A01297231
Jan 31st, 2026

What it does

Goal

Use a controller/worker architecture to create a distributed password cracking system for UNIX systems.

Controller

Accept command-line input for the controller: shadow file (file to copy), username, and port number. Uses the inputs to identify hashing algorithms, salt, and password hash and registers with a single worker to crack the password. Then receives results to report timing and results. The controller will also send heartbeat requests to verify the workers status.

Worker

Accept command-line input for the worker: controller_host (hostname/IP address), and port number. Then registers with a controller and receives a cracking job. The worker will then crack the password with a given number of threads. After cracking, send results to the controller. The worker will also respond to heartbeat requests.

Success condition (observable outcome)

The worker must successfully crack the password, and the program must measure and report controller-side parsing time, job dispatch latency, worker cracking time, result return latency, and total end-to-end runtime. The program must then exit successfully with no error messages.

Invariant (one checkable rule)

The program never displays a message unless the input arguments are valid.

How I will test it

Valid Shadow File and Username

The program is invoked from the command line with an existing and valid shadow file and username

Steps:

1. Run the program with valid arguments
2. Observe standard output.
3. Observe program termination.

Expected result:

- The controller registers a worker successfully retrieves the cracked password and timing results. Then the program exits successfully.

What it proves:

- Valid arguments transitions the system from argument handling to processing to connecting/connected, retrieving results, displaying results, and then to clean termination.

Invalid Shadow File or Username

The program is invoked from the command line with a non-existing shadow file or username

Steps:

1. Run the program with a non-existing shadow file or username
2. Observe standard error output.
3. Observe program termination.

Expected result:

- The controller fails to locate the shadow file and password information from the inputted shadow file and username

What it proves:

- The controller requires an existing shadow file and username

Missing Arguments

The controller program or worker program is invoked with a missing argument

Steps:

1. Run the program with a missing argument
2. Observe standard error output.
3. Observe program termination.

Expected result:

- An error or usage message is displayed; The program exits after cleanup.

What it proves:

- The system detects missing required inputs and prevents progression to the processing state.

Controller Invalid Arguments

The program is invoked with a non-numeric, or negative port.

Steps:

1. Run the program with an invalid port value.
2. Observe standard error output.
3. Observe program termination.

Expected result:

- An error or usage message is displayed, and the program exits after cleanup.

What it proves:

- Invalid numeric input triggers an error transition, blocking successful execution.

Worker Invalid Arguments

The program is invoked with a non-numeric, or negative port. Or the program is invoked with an invalid IP address

Steps:

1. Run the program an invalid port value or IP address.
2. Observe standard error output.
3. Observe program termination.

Expected result:

- An error or usage message is displayed, and the program exits after cleanup.

What it proves:

- Invalid numeric input triggers an error transition, blocking successful execution.

Worker Host Not Found

The program is invoked with valid arguments, but the IP address will not be a working IP address.

Steps:

1. Run the program with valid arguments without running a host program.
2. Wait until the timeout timer ends.

3. Observe program termination.

Expected result:

- An error is displayed, and the program exits after cleanup.

What it proves:

- The worker requires a listening host to connect to.

Help Option

The program is invoked with a help option.

Steps:

1. Run the program with the help flag.
2. Observe standard output.
3. Observe program termination.

Expected result:

- Usage information is displayed, and the program exits without attempting to process or display messages.

What it proves:

- Explicit user requests for help bypass normal execution and terminate cleanly.

FSM

State meanings

Controller

State	Meaning
START	The program has begun execution, but no work has been performed
PARSE_ARGS	Raw input arguments are being read and identified
HANDLE_ARGS	Parsed arguments are being validated and converted
USAGE	An error or misuse has been detected, and feedback is shown

READ_SHADOW	Shadow file is opened and read
PARSE_SHADOW	Shadow entry is parsed to extract hashing algorithm, salt, and password hash
INIT_LISTENER	Controller initializes communication endpoint for worker registration
WAIT_FOR_WORKER	Controller waits for a single worker to register
REGISTER_WORKER	Worker registration is accepted and acknowledged
DISPATCH_JOB	Full cracking job (hash, salt, algorithm) is sent to the worker
WAIT_FOR_RESULT	Controller blocks while awaiting worker result
RECEIVE_RESULT	Results are received from the dispatched job
REPORT_RESULTS	Timing statistics and final outcome are displayed
SEND_HEARTBEAT	Send a heartbeat request
RECEIVE_HEARTBEAT	Receive the heartbeat back
CLEANUP	Cleanup the program
EXIT	The program has terminated

Worker

State	Meaning
START	The program has begun execution, but no work has been performed
PARSE_ARGS	Raw input arguments are being read and identified
HANDLE_ARGS	Parsed arguments are being validated and converted
USAGE	An error or misuse has been detected, and feedback is shown
INIT_SERVICE	Worker initializes its service environment
INIT_THREADS	Initializes the threads
REGISTER_CONTROLLER	Worker registers itself with the controller
WAIT_FOR_JOB	Worker waits for a cracking job assignment
ASSIGN_JOBS	Splits up the work assigned to different threads

INIT_CRACKING	Cracking context is initialized (wordlist, rules, hash function)
ATTEMPT_CRACK	Each candidate is checked against the target hash
CHECK_MATCH	Check if the cracked password is correct
WORK_EXHAUSTED	Failed to crack password
RECEIVE_HEARTBEAT	Receives a heartbeat request
SEND_HEARTBEAT	Sends a heartbeat back
UPDATE_PROGRESS	Updates the workers progress
SEND_SUCCESS	Cracked password is sent back to the controller
SEND_FAILURE	Failure result is sent back to the controller
CLEANUP	Cleanup the program
EXIT	The program has terminated

Transition table

Controller

State	Call (function)	Fail state	Success state
START	parse_arguments		PARSE_ARGS
PARSE_ARGS	handle_arguments	USAGE	HANDLE_ARGS
PARSE_ARGS	usage		USAGE
HANDLE_ARGS	read_shadow	USAGE	READ_SHADOW
HANDLE_ARGS	usage		USAGE
USAGE	cleanup		CLEANUP
CLEANUP			EXIT
READ_SHADOW	parse_shadow	USAGE	INIT_LISTENER
READ_SHADOW	usage		USAGE

INIT_LISTENER	wait_worker	USAGE	REGISTER_WORKER
INIT_LISTENER	usage		USAGE
REGISTER_WORKER	dispatch_job	USAGE	DISPATCH_JOB
REGISTER_WORKER	usage		USAGE
DISPATCH_JOB	wait_result	USAGE	WAIT_FOR_RESULT
DISPATCH_JOB	usage		USAGE
WAIT_FOR_RESULT	receive_result	USAGE	RECEIVE_RESULTS
WAIT_FOR_RESULT	usage		USAGE
WAIT_FOR_RESULT	receive_result	USAGE	RECEIVE_RESULTS
RECEIVE_RESULTS	report_results	USAGE	REPORT_RESULTS
RECEIVE_RESULTS	usage		USAGE
REPORT_RESULTS	cleanup		CLEANUP

Worker

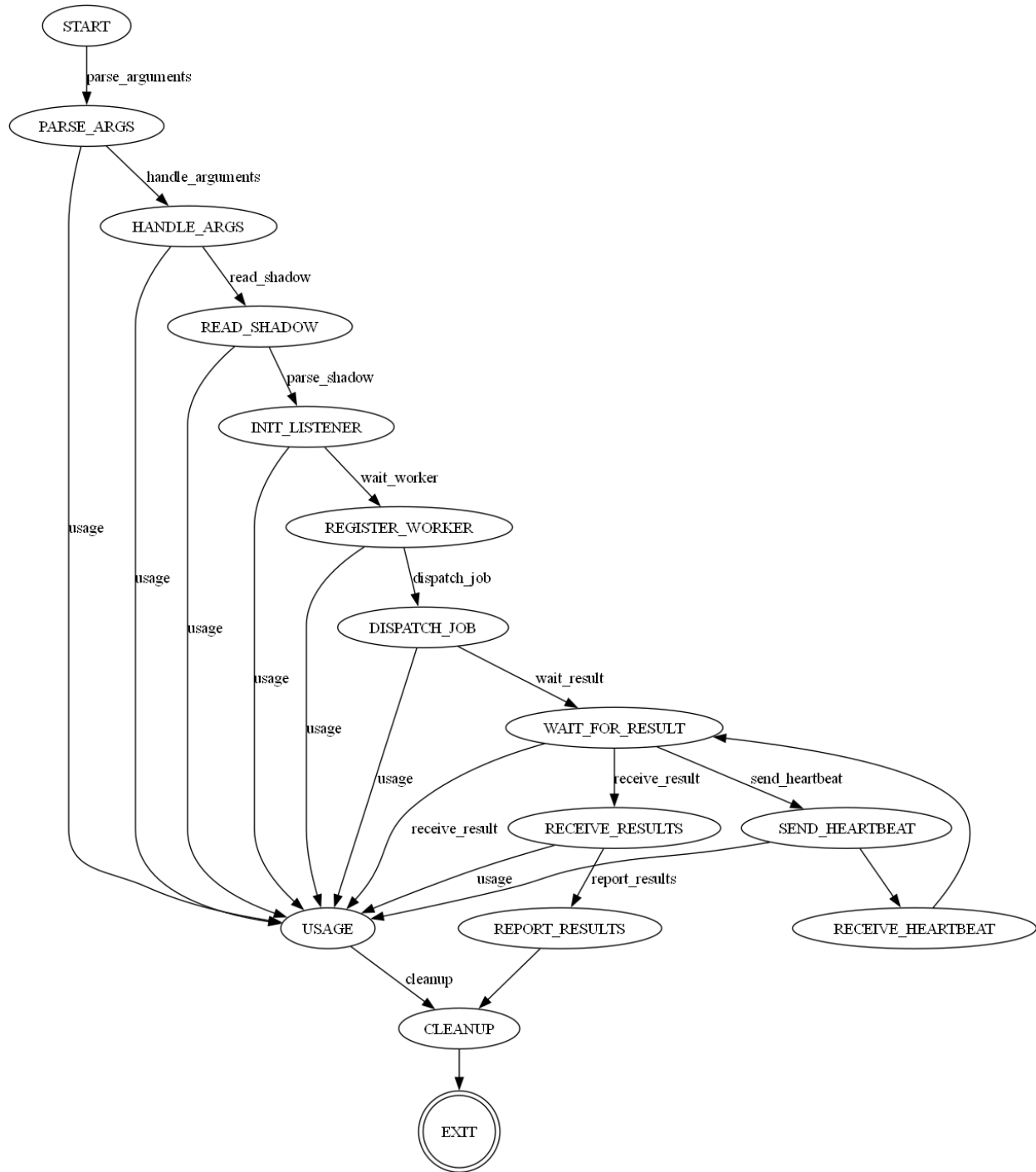
State	Call (function)	Fail state	Success state
START	parse_arguments		PARSE_ARGS
PARSE_ARGS	handle_arguments	USAGE	HANDLE_ARGS
PARSE_ARGS	usage		USAGE
HANDLE_ARGS	init_service	USAGE	INIT_SERVICE
HANDLE_ARGS	usage		USAGE
USAGE	cleanup		CLEANUP
CLEANUP			EXIT
INIT_SERVICE	init_threads	USAGE	INIT_THREADS
INIT_SERVICE	usage		USAGE
INIT_THREADS	register_controller	USAGE	REGISTER_CONTROLLER

INIT_THREADS	usage		USAGE
REGISTER_CONTROLLER	wait_job	USAGE	WAIT_FOR_JOB
REGISTER_CONTROLLER	usage		USAGE
WAIT_FOR_JOB	assign_jobs	USAGE	ASSIGN_JOBS
WAIT_FOR_JOB	usage		USAGE
ASSIGN_JOBS	start_cracking	USAGE	INIT_CRACKIGN
ASSING_JOBS	usage		USAGE
INIT_CRACKING	crack_password	USAGE	ATTEMPT_CRACK
INIT_CRACKING	usage		USAGE
ATTEMPT_CRACK	check_match	USAGE	CHECK_MATCH
ATTEMPT_CRACK	usage		USAGE
ATTEMPT_CRACK	update_progress	USAGE	UPDATE_PROGRESS
CHECK_MATCH	success	USAGE	SEND_SUCCESS
CHECK_MATCH	crack_password	USAGE	ATTEMPT_CRACK
CHECK_MATCH	usage		USAGE
ATTEMPT_CRACK	exhausted	USAGE	WORK_EXHAUSTED
ATTEMPT_CRACK	usage		USAGE
WORK_EXHAUSTED	failure	USAGE	SEND_FAILURE
WORK_EXHAUSTED	usage		USAGE
SEND_SUCCESS	cleanup	USAGE	CLEANUP
SEND_SUCCESS	usage		USAGE
SENDS_FAILURE	cleanup	USAGE	CLEANUP
SENDS_FAILURE	usage		USAGE
RECEIVE_HEARTBEAT	send_heartbeat	USAGE	SEND_HEARTBEAT
RECEIVE_HEARTBEAT	usage		USAGE
SEND_HEARTBEAT	usage		USAGE

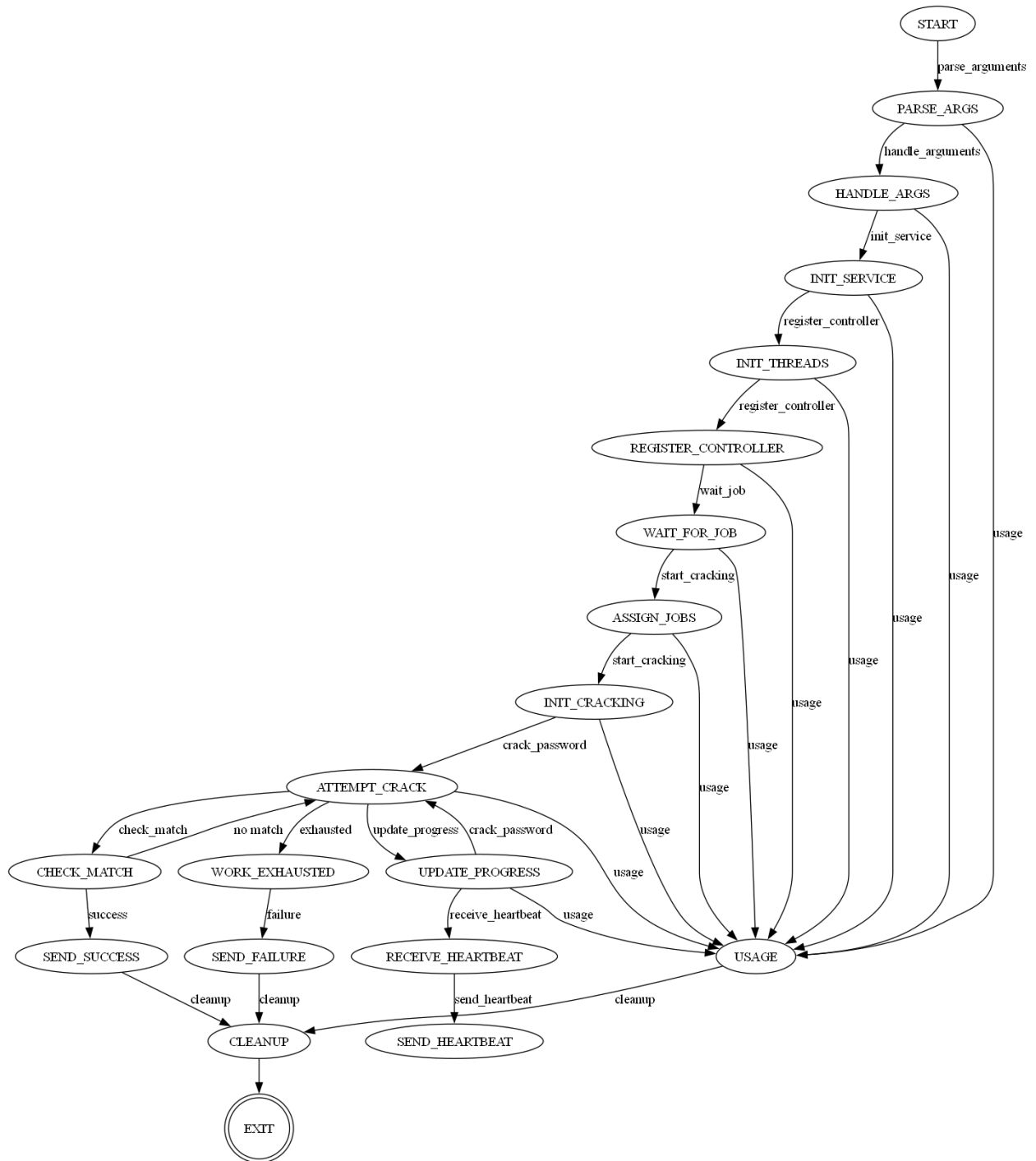
UPDATE_PROGRESS	crack_password	USAGE	ATTEMPT_CRACK
UPDATE_PROGRESS	receive_heartbeat	USAGE	
UPDATE_PROGRESS	usage		USAGE

FSM diagram

Controller



Worker



Functions

Controller

Name	Inputs	Outputs	Responsibility
parse_arguments	Raw command-line input	Parsed argument data	Identify options and message input
handle_arguments	Parsed argument data	Validated settings or error	Ensure inputs are valid and usable
usage	Error context	User-facing message	Inform the user of correct usage
read_shadow	Shadow file, username	Shadow file information	Opens and reads the shadow file
cleanup	Program context	Exit status	Ensure orderly termination
parse_shadow	Shadow file, username	Hashing algorithm, salt, password hash	Extracts the information from the shadow file
wait_worker		Connected worker	Connects a worker to the controller
dispatch_job	Hashing algorithm, salt, password hash		Sends cracking information to worker
wait_result		Worker results	Waits for a return from the worker
receive_result	Worker results	Cracking results	Receives the cracked password results
report_result	Cracking results	Results report	Gives a report based on the cracking results
send_heartbeat			Send a heartbeat request to the worker
receive_heartbeat		Worker progress	Receives the heartbeat response from the worker

Worker

Name	Inputs	Outputs	Responsibility
parse_arguments	Raw command-line input	Parsed argument data	Identify options and message input
handle_arguments	Parsed argument data	Validated settings or error	Ensure inputs are valid and usable
usage	Error context	User-facing message	Inform the user of correct usage
init_service		Connection service started	Starts up the socket to connect
cleanup	Program context	Exit status	Ensure orderly termination
register_controller	IP address, port	Established connection	Establishes a connection with the controller
wait_job		Cracking job	Waits for a job from the controller
start_cracking	Hashing algorithm, salt, hashed password		Initialize cracking content
crack_password	Hashing algorithm, salt, hashed password	Cracked results	Attempts to crack the password
check_match	Hashed password, cracked results	Cracked success or failure	Checks if the cracked results matches the password
success	Cracked results	Sends success results	Establishes successfully cracking
exhausted		Displays failure results	Work has been exhausted and cracking failed
failure		Sends failure results	

receive_heartbeat			Receives the heartbeat request from the controller
send_heartbeat	Worker progress		Sends the work progress to the controller